

Q1 [40 points] Use APIs to Build a Network of Illegal Wildlife Trafficking

The illegal wildlife trade (IWT) is one of the most lucrative forms of transnational organized crime, with air travel playing a key role in global wildlife smuggling networks. In this question, you will use data drawn from observed wildlife trafficking incidents to build, annotate, and analyze a network of airports and the flights connecting them. You will first construct the full flight network, representing all major/regional airports and the flights connecting them, and then build a sub-IWT network that only includes airports and flight paths with documented/reported wildlife trafficking incidents.

In this question, you will use APIs to access a dataset provided by TRAFFIC, an NGO working to ensure the global wildlife trade is legal and sustainable. TRAFFIC provides data through its interactive Wildlife Trade Portal, some of which we have already downloaded and pre-processed for you to utilize in Q1.

Technology	• Python 3.10.x
Allowed Libraries	The Python Standard Library and Requests only .
Max runtime	5 minutes. Submissions exceeding this will receive zero credit .
Deliverables	• Q1.py: The completed Python file • full_centrality.csv: The CSV file containing the degree centrality of all airports in the full flight network. • iwt_centrality.csv: The CSV file containing the degree centrality in the sub-IWT flight network. • cleaned_iwt.csv: The CSV file containing each unique flight leg with observed wildlife trafficking activity.

Follow the instructions found in `Q1.py` to complete the `Graph` class and the other global functions. The `Graph` class will serve as a re-usable way to represent and write out your collected graph data between the full flight and sub-IWT networks. The global functions will be used to connect to the API, clean the wildlife trafficking flight data, and write your results to CSVs.

API Setup

If you are new to Docker, complete the setup steps below before starting the Q1 API.

1. Install Docker Desktop for your operating system.

- Windows 10/11:** Download and install [Docker Desktop for Windows](#). Use the WSL 2 option when prompted. If Docker reports that WSL is missing or out of date, follow the [Microsoft WSL installation instructions](#), restart your computer if prompted, and then reopen Docker Desktop. Docker must be using Linux containers; this is the default when using the WSL 2 engine.
- macOS:** Download and install [Docker Desktop for Mac](#). Choose the Apple silicon download if *About This Mac* lists an Apple chip (such as M1, M2, M3, or M4); choose the Intel download if it lists an Intel processor. Open Docker Desktop after installation and approve any permissions requested by macOS.

2. Start Docker Desktop.

Wait until Docker Desktop reports that the Docker engine is running. Keep Docker Desktop open while you work on Q1.

3. Open a terminal.

- On Windows, open **PowerShell** from the Start menu. Do not use Command Prompt (CMD) for the commands in this guide.
- On macOS, open the **Terminal** application from *Applications* → *Utilities*.

4. Confirm that Docker is ready.

Run both commands below. The first prints the installed Docker version. The second should print information about the Docker engine.

```
docker --version
docker info
```

If `docker info` reports that it cannot connect to the Docker daemon or Docker engine, open Docker Desktop and wait until it finishes starting, then try the command again.

5. **Start the Q1 API using the commands below in this section.** The `docker pull` command downloads the Q1 API image. The `docker run` command starts the API and connects port 3000 in the container to port 3000 on your computer. The first download may take several minutes depending on your internet connection.

When `docker run` is successful, it remains active and displays API output in that terminal. **Leave that terminal window open while running Q1.py.** Open a second PowerShell or Terminal window to run your Python code. Your Python code runs on your computer, not inside the Docker container.

6. **Stop and restart the API.** Press `Ctrl+C` in the terminal running the API to stop it. To use the API in a later work session, start Docker Desktop and run the `docker run` command again. You do not need to repeat `docker pull` unless the image has been updated or removed from your computer.

7. **Troubleshoot common problems.**

- If you see an error that port 3000 is already allocated or in use, run `docker ps`. If the Q1 image is already running with a port mapping that includes 3000, first check <http://localhost:3000/docs>; you may already use that running API. Otherwise, stop the container occupying the port with `docker stop <CONTAINER ID>` and run the Q1 `docker run` command again.
- If <http://localhost:3000/docs> does not load, confirm that Docker Desktop is running and that the terminal containing `docker run` has not been closed or stopped. Check that the terminal does not display an error.
- In Windows PowerShell, use `curl.exe` instead of `curl` if PowerShell handles the test commands differently than expected. Opening <http://localhost:3000/docs> in a browser is also sufficient to confirm that the API is running.

The data for this question is served through a locally hosted REST API using Docker. You must use the `get_data()` function in the skeleton file to make all API requests. To start the API, ensure Docker is installed on your machine and run the following two commands in your terminal:

```
docker pull polodataclub/cse6242hw1-public:q1-api
docker run -p 3000:3000 polodataclub/cse6242hw1-public:q1-api
```

Leave the container running in that terminal window. You can verify the API is working and explore the full documentation at <http://localhost:3000/docs> in your browser. You can also test each endpoint directly using `curl`:

```
curl http://localhost:3000/airports
curl http://localhost:3000/flights
curl http://localhost:3000/iwt_flights
```

Endpoint	Description
<code>/airports</code>	Returns a list of ~2,000 major and regional airports worldwide. Each record includes the airport's IATA code (unique three-letter codes that identify airports globally, like ATL), name (e.g., Hartsfield-Jackson Atlanta International Airport), city, country, latitude, longitude, and <code>iwt_incidents_observed_count</code> , the number of observed wildlife trafficking incidents at that airport from our data providers.
<code>/flights</code>	Returns a list of ~14,000 unique commercial flight routes. Each record includes <code>airport_a</code> and <code>airport_b</code> as IATA codes. This data is undirected: a record connecting <code>airport_a</code> and <code>airport_b</code> implies there is also a flight connecting <code>airport_b</code> to <code>airport_a</code> .
<code>/iwt_flights</code>	Returns a list of ~500 documented IWT incidents. Each record contains at least 2, but up to 5 airport IATA codes (<code>airport_a</code> through <code>airport_e</code>) representing the sequential stops of one documented trafficking incident. The scarcity of this data highlights how challenging it is to capture robust and representative wildlife trafficking path data. Unused stop columns are returned as <code>None</code> .

Tasks and point breakdown

1. [10 pts] Implement the `Graph` class according to the instructions in `Q1.py`.

- Implement `add_node()`. This function will add nodes to the network. In our case, these are the airports from the `/airports` endpoint. Use the 'IATA' and 'name' fields from this dataset. Duplicate nodes must be ignored.

- b. Implement `add_edge()`. This function will add the edges to the network. In our case, these are the flights from the `/flights` and `/iwt_flights` endpoints. The graph should be treated as undirected (e.g. `ATL → JNB` is the same edge as `JNB → ATL`). Duplicate edges must be ignored.
 - c. Implement `degree centrality()`. This function will calculate the normalized degree centrality of all nodes/airports in the full flight and sub-IWT flight networks. The degree $d(v)$ of a node v is number of edges attached to a node v . A node v with degree $d(v)$ has a normalized degree centrality equal to $\text{degree_centrality}(v) = d(v)/(N - 1)$, where N is the number of airports added to the graph with `add_node()`. Return a score for every airport that appears in the graph — including airports with degree 0, and any airport that appears only as an endpoint of an edge. Outputs should be rounded to 6 decimal places using the Python `round()` function.
2. [3 pts] Implement the `get_data()` global function. Follow the directions in the 'API Setup' section above.
3. [3 pts] Implement the `clean_trafficking_paths()` global function.
 - a. Each itinerary is a list of at least 2, but up to 5 IATA codes. `None` values and empty strings represent unused stop columns and should be ignored.
 - b. Extract every consecutive pair of adjacent airports in order. For example,
 - `['CPH', 'VIE', None, None, None] → [('CPH', 'VIE')]`
 - `['LAX', 'DTW', 'IST', 'PVG', None] → [('LAX', 'DTW'), ('DTW', 'IST'), ('IST', 'PVG')]`
 - c. Duplicate pairs that appear across multiple itineraries must only be included once in the output (e.g. `('CPH', 'VIE')` should only appear once in the output).
 - d. Preserve the order in which stops appear. `(A,B)` and `(B,A)` are distinct flight legs in `cleaned_iwt.csv`, even though `add_edge()` treats them as the same undirected edge.
 - e. The returned list must be sorted alphabetically by the first IATA code, with ties broken alphabetically by the second.
4. [3 pts] Implement the `write_centrality_file()` global function. This function writes degree centrality scores to a CSV file.
 - a. The header row must be: `iata,degree centrality`
 - b. Rows must be sorted in descending order by centrality score.
 - c. Ties must be broken alphabetically by IATA code (ascending).
 - d. Double check centrality outputs from `degree centrality()` are rounded to 6 decimal points using the Python `round()` function.
5. [21 pts] Build both graphs and compute degree centrality in `__main__` according to the step-by-step comments in `Q1.py`.
 - a. **Step 1: Load airports.** Call `get_data('/airports')` and add every airport as a node to `full_graph`.
 - b. **Step 2: Load full flight edges.** Call `get_data('/flights')` and add every route as an edge to `full_graph`. `add_edge()` should handle deduplication automatically.
 - c. **Step 3: Load IWT nodes.** Using the airports response already retrieved in Step 1 (do not make another API call), add only airports with a non-zero value for `iwt_incidents_observed_count` as nodes to `iwt_graph`. This ensures that airports that have observed IWT activity but may not have documented inbound or outbound flights are still included in the IWT network.
 - d. **Step 4: Load and clean IWT edges.** Call `get_data('/iwt_flights')`. The API returns a list of dicts; convert these to a list of lists before passing to `clean_trafficking_paths()`. Add each unique cleaned pair as an edge to `iwt_graph`. Write the cleaned pairs to `cleaned_iwt.csv` with header `iata_a,iata_b`.
 - e. **Step 5: Compute and write degree centrality.** Call `degree centrality()` on both `full_graph` and `iwt_graph` and write the results using `write_centrality_file()`.
 - **full_centrality.csv:** degree centrality for the full flight network
 - **iwt_centrality.csv:** degree centrality for the IWT sub-network

Q2 [30 points] SQLite

SQLite is a lightweight, serverless, embedded database that can easily handle multiple gigabytes of data. It is one of the world's most popular embedded database systems. It is convenient to share data stored in an SQLite database — just one cross-platform file that does not need to be parsed explicitly (unlike CSV files, which must be parsed). You can find instructions to install SQLite [here](#). In this question, you will construct a database in SQLite, partition it, and combine information within tables to answer questions.

In this question, you will work with a dataset provided by **TRAFFIC**, an NGO working to ensure the global wildlife trade is legal and sustainable. TRAFFIC provides data through their interactive Wildlife Trade Portal, some of which we have already downloaded and pre-processed for you to utilize in Q2. You will modify the given `Q2.py` file by adding SQL statements to it. We suggest testing your SQL locally on your computer using interactive tools to speed up testing and debugging, such as DB Browser for SQLite.

Technology	• SQLite release 3.37.2 • Python 3.10.x
Allowed Libraries	Do not modify import statements. Everything you need to complete this question has been imported for you. Do not use other libraries for this question.
Max runtime	10 minutes. Submissions exceeding this will receive zero credit .
Deliverables	• Q2.py: Modified file containing all the SQL statements you have used to answer parts a - h in the proper sequence.

Important Notes

- If the **final** output asks for a decimal column, round it to 2 decimal places using the `ROUND()` SQL function.
- A sample class has been provided to show example SQL statements; you can turn off this output by changing the global variable `SHOW` from `True` to `False`.
- In this question, you must only use **INNER JOIN** when performing a join between two tables. Other types of joins may result in incorrect results.

Tasks and point breakdown

1. [6 pts] Create tables and import data.

- a. [3 pts] Create three tables named `incidents`, `details` and `outcomes` with columns having the indicated data types:

i. `incidents`

- `report_id` (text)
- `category` (text)
- `date` (text)

ii. `details`

- `report_id` (text)
- `subject` (text)
- `transport_mode` (text)
- `detection` (text)

iii. `outcomes`

- `report_id` (text)
- `outcome` (text)
- `num_ppl_fined` (integer)
- `fine` (real)
- `num_ppl_arrested` (integer)
- `prison_time` (real)

- `prison_time_unit` (text)

- [3 pts]** Import the three provided `.csv` files into the three tables created above.
 - Insert `incidents.csv` into the `incidents` table.
 - Insert `details.csv` into the `details` table.
 - Insert `outcomes.csv` into the `outcomes` table.

Write Python code that imports the `.csv` files into the individual tables. This will include looping through the file and using the `INSERT INTO` SQL command. You **must** only use relative paths to the Q2 directory while importing files, since absolute/local paths are specific locations that exist only on your computer; the auto-grader cannot access such locations and will fail.

- [1 pt]** *Create indexes.* Create the following indexes. Indexes increase data retrieval speed; though the speed improvement may be negligible for this small database, it is significant for larger databases.

- `incident_index` for the `report_id` column in the `incidents` table
- `detail_index` for the `report_id` column in the `details` table
- `outcome_index` for the `report_id` column in the `outcomes` table

- [2 pts]** *Calculate a percentage.* Find the percentage of incidents that occurred between January 1st, 2018, and December 31st, 2020 (inclusive).

- Output format and example value (*percentage*):

50.41

- [3 pts]** *Find the most common transport modes.* Identify the three most common transport modes that were detected by intelligence. Filter the detection column in the details table where the value is "Intelligence". Sort by most common transport mode to least common. Do not include null or empty transport modes.

- Output format and example row values (`transport_mode`, `count`):

Sea 15

- [4 pts]** *Identify detection methods with high arrest rates.* Identify the three detection methods with the highest number of average arrests across incidents. Only include incidents with one or more arrests in the average calculation. Only include detection methods with at least 100 incidents (with one or more arrests) in the final list. Sort by highest average to lowest.

- Output format and example row values (`detection`, `count`, `avg_ppl_arrested`):

Dogs 15 4.21

- [4 pts]** *List categories with the longest prison sentences.* Identify the incident categories with the highest average prison sentences in days. Use the `prison_time_unit` column to convert `prison_time` into days if necessary (assume 365 days in a year, 30 in a month, 7 in a week). If `prison_time_unit` is N/A, then set `prison_time` to 0 days. Only include incident categories with more than 50 incidents. Sort by highest to lowest average.

- Output format and example row values (`category`, `count`, `avg_prison_time_days`):

"1. Seizure" 52 365.88

Do not add quotes to your output; they are included in the example output to signify that "1. Seizure" is a complete string.

- [4 pts]** *Creating a view.*

- [2 pts]** *Create a view (virtual table)* called `finest` that lists report IDs, dates, the number of people fined, and the fine amount. Only include incidents where at least one person was fined. The view should have the following columns:

- `report_id`
- `date`
- `num_ppl_fined`
- `fine`

Remember that creating a view will not produce any output, so you should test your view with a few simple select statements during development. One such test has already been added to the code as part of the auto-grading.

- [2 pts]** *Find years with the highest fine amounts.* Using the `finest` view, identify the 3 years with the most total fines given out and how many people they fined. Sort by highest to lowest fine amount.

- Output format and example row values (`year`, `total_ppl_fined`, `total_fine_amount`):

2017 300 8000000.55

Optional Reading: [Why create views?](#)

8. [6 pts] FTS

SQLite supports simple but powerful Full Text Search (FTS) for fast text-based querying ([FTS documentation](#)).

a. Create a virtual table called `incident_overviews` using `fts5` with the following columns

- `report_id`
- `subject`

NOTE: If you have issues that FTS is not enabled, try the following steps

- Go to sqlite3 downloads page: <https://www.sqlite.org/download.html>
- Download the dll file for your system
- Navigate to your Python packages folder,
e.g., `C:\Users...\Anaconda3\pkgs\sqlite-3.29.0-he774522_0\Library\bin`
- Drop the downloaded .dll file in the bin.

b. Insert data from the `report_id` and `subject` columns in the `details` table into `incident_overviews`.

c. Count the number of incidents that contain the terms 'dead' and 'pangolin' in the `subject` text with no more than two intervening terms between them. Matches are not case sensitive and should be full words, not substrings.

Example:

Allowed: "1 dead Sunda pangolin"

Disallowed: "1 live pangolin seized and 2 dead birds found".

- Output format and example (`count`):

15

Q3 [15 points] D3 Warmup: Visualizing Wildlife Trafficking by Species

Using species-related data from the Wildlife Trade Portal, you will build a bar chart to visualize the most frequently illegally trafficked species between 2015 and 2023. Using D3, you will get firsthand experience with how interactive plots can make data more visually appealing, engaging, and easier to parse.

Read chapters 4–8 of Scott Murray’s [Interactive Data Visualization for the Web, 2nd edition](#) (sign in using your GT account, e.g., jdoe3@gatech.edu). This reading provides an important foundation you will need for Homework 2. The question and autograder have been developed and tested for D3 version 5 (v5), while the book covers v4. What you learn from the book is transferable to v5, as v5 introduced few breaking changes. We also suggest briefly reviewing chapters 1–3 for background information on web development.

TRAFFIC International (2025) Wildlife Trade Portal. Available at www.wildlifetradeportal.org.

Technology	• D3 Version 5 (included in the lib folder) • Chrome 97.0 (or newer): the browser for grading your code • Python HTTP server (for local testing)
Allowed Libraries	D3 library is provided to you in the lib folder. You must NOT use any D3 libraries (<code>d3*.js</code>) other than the ones provided.
Deliverables	• Q3.html: Modified file containing all HTML, JavaScript, and any CSS code required to produce the bar plot. Do not include the D3 libraries or <code>q3.csv</code> dataset.

Important:

- Setup an HTTP server to run your D3 visualizations as discussed in the D3 lecture (OMS students: [watch lecture video](#). Campus students: see [lecture PDF](#)). The easiest way is to use [http.server](#) for Python 3.x. Run your local HTTP server in the hw1-skeleton/Q3 folder.
- We have provided sections of skeleton code and comments to help you complete the implementation. While you do not need to remove them, you need to write additional code to make things work.
- All `d3*.js` files are provided in the lib folder and referenced using relative paths in your HTML file. For example, since Q3 requires reading data from the `q3.csv` file, the path must be “`q3.csv`” and **NOT** an absolute path such as “`C:\Users\...\q3.csv`”. Absolute paths are specific locations that exist only on your computer, which means your code will **NOT** run on our machines when we grade, and you will lose points. As file paths are case-sensitive, ensure you correctly provide the relative path.
- Load the data from `q3.csv` using D3 fetch methods. We recommend `d3.dsv()`. Handle any data conversions that might be needed, e.g., strings that need to be converted to integer. See <https://github.com/d3/d3-fetch#dsv>.
- **VERY IMPORTANT:** Use the [Margin Convention](#) guide to specify chart dimensions and layout.

Tasks and point breakdown

Q3.html: When run in a browser, should display a **horizontal bar plot** with the following specifications:

1. **[3.5 pts]** The bar plot must display one bar for each of the five most trafficked species by count. Each bar’s length corresponds to the number of wildlife trafficking incidents involving that species between 2015 and 2023, represented by the “count” column in our dataset.
2. **[1 pt]** The bars must have the same fixed thickness, and there must be some space between the bars, so they do not overlap.
3. **[3 pts]** The plot must have visible X and Y axes that scale according to the generated bars. That is, the axes are driven by the data that they are representing. They must **not** be hard-coded. The x-axis must be a `<g>` element having the id: “`x_axis`” and the y-axis must be a `<g>` element having the id: “`y_axis`”.
4. **[2 pts]** Set x-axis label to “Count” and y-axis label to “Species”. The x-axis label must be a `<text>` element having the id: “`x_axis_label`” and the y-axis label must be a `<text>` element having the id: “`y_axis_label`”.

5. **[2 pts]** Use a linear scale for the X-axis to represent the count (recommended function: `d3.scaleLinear()`). Only display ticks and labels at every 500 interval. The x-axis must be displayed below the plot.
6. **[2 pts]** Use a categorical scale for the Y-axis to represent the species names (recommended function: `d3.scaleBand()`). Order the species names from greatest to least on “Count” and limit the output to the top 5 species. The y-axis must be displayed to the left of the plot.
7. **[1 pt]** Set the HTML title tag and display a title for the plot. Those two titles are independent of each other and need to be set separately. Set the HTML title tag (i.e., `<title> Wildlife Trafficking Incidents per Species (2015 to 2023)</title>`). Position the title “Wildlife Trafficking Incidents per Species (2015 to 2023)” above the bar plot. The title must be a `<text>` element having the id: “title”.
8. **[0.25 pt]** Add your GT username (usually includes a mix of letters and numbers) to the area beneath the bottom-right of the plot. The GT username must be a `<text>` element having the id: “credit”.
9. **[0.25 pt]** Fill each bar with a unique color. We recommend using a [colorblind-safe palette](#).

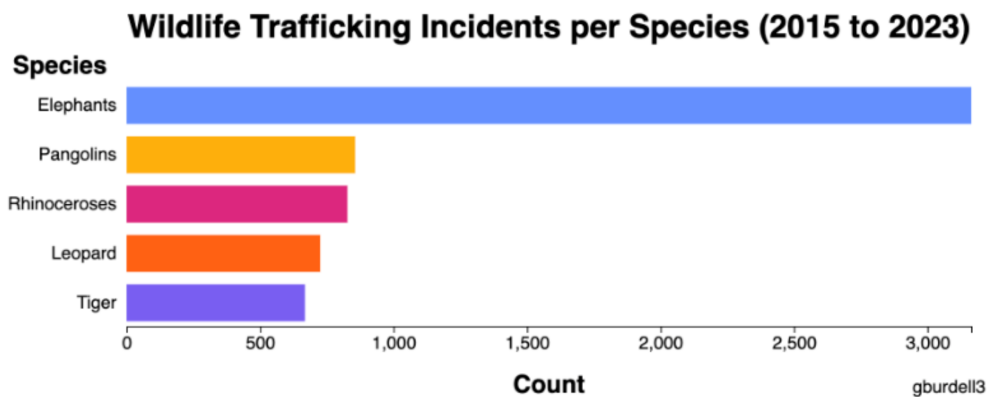


Figure 1: Example bar plot for wildlife trafficking incidents per species (2015 to 2023). Your design does not need to replicate this exact plot.

NOTE: Gradescope will render your plot using Chrome and present you with a Dropbox link to view the screenshot of your plot as the autograder sees it. This visual feedback helps you adjust and identify errors, e.g., a blank plot indicates a serious error. Your design does not need to replicate the solution plot. However, **the autograder requires the following DOM structure (including using correct IDs for elements) and sizing attributes to know how your chart is built.**

```
<svg id="svg1"> plot
| width: 900
| height: 370
|
+-- <g id="container"> containing Q3.a plot elements
|
|   +-- <g id="bars"> containing bars
|   |
|   +-- <g id="x_axis"> x-axis
|   | |
|   | +-- (x-axis elements)
|   |
|   +-- <text id="x_axis_label"> x-axis label
|   |
|   +-- <g id="y_axis"> y-axis
|   | |
|   | +-- (y-axis elements)
|   |
|   +-- <text id="y_axis_label"> y-axis label
|   |
|   +-- <text id="credit"> GTUsername
|   |
|   +-- <text id="title"> chart title
```

Q4 [10 points] OpenRefine

OpenRefine is a powerful tool for working with messy data, allowing users to clean and transform data efficiently. Use OpenRefine in this question to clean data around wildlife trafficking incidents. Construct GREL queries to identify incidents that occurred in airports.

OpenRefine is a Java application that requires Java JRE to run. However, OpenRefine v.3.6.2 comes with a compatible Java version embedded with the installer. So, there is no need to install Java separately when working with this version. Go through the main features on [OpenRefine's homepage](#). Then, [download and install](#) OpenRefine 3.6.2. The link to release 3.6.2 is <https://github.com/OpenRefine/OpenRefine/releases/tag/3.6.2>.

Technology	• OpenRefine 3.6.2
Deliverables	• Q4.csv: Export the final table as a CSV file. • history.json: Submit a list of changes made to file in JSON format. Go to 'Undo/Redo' Tab → 'Extract' → 'Export'. This downloads <code>history.json</code>.

Tasks and point breakdown

Import Dataset

- a. [Run](#) OpenRefine and point your browser at <https://127.0.0.1:3333>.
 - b. Choose “Create Project” → This Computer → `incidents.csv`. Click “Next”.
 - Don't open the csv in Excel/Numbers as they may change the formatting of the data.
 - c. You will now see a preview of the data. Click “Create Project” at the upper right corner.

1. [3 pts] Filter and Remove Data

- a. Use one [facet](#) to remove [blank](#) (null or empty) values from the “Common Name” column.
- b. Use one [text filter](#) to remove any rows that do not mention “Airport” (case sensitive) in the “Subject” column.

2. [7 pts] Identify Airports

- a. Create a column named “Airport” based on the “Subject” column that lists the full airport name mentioned.
 - i. For example, a “Subject” value of “77kg of sandalwood at the Chhatrapati Shivaji Maharaj International Airport, one Sudanese national” should create an “Airport” value of “Chhatrapati Shivaji Maharaj International Airport”.
 - ii. There may not be a way to parse all airport names 100% correctly. Try to identify a heuristic that works in most cases.
 - iii. There should not be any blank values in the “Airport” column. Adjust your heuristic if it's a common issue. If there are rare edge cases, you can [manually edit](#) the offending cells.
 - iv. Sample Resources: [Regular Expressions](#), [GREL functions](#).
- b. Cluster and merge similar airport names in the “Airport” column. Select the “Airport” column, go to “Edit cells”, then “Cluster and edit”. Explore the various keying functions under the “key collision” method and use the resulting clusters to merge variations of the same airport name.
 - For example, “Detroit International Airport” and “Detroit Airport” can be merged into “Detroit International Airport”. The value you choose to merge into does not matter – you can choose to merge into either “Detroit International Airport” or “Detroit Airport”, but both values should not be present.
 - If you have airport names that appear to match what Gradescope expects but are not passing the test case, check that your airport names do not include any hidden characters.

Q5 [5 points] Introduction to Python Flask

In this question, you will build a web application using Flask. [Flask](#) is a lightweight web application framework written in Python that provides you with tools, libraries, and technologies to build a web application quickly and scale it up as needed. The website will display wildlife trafficking data and allow users to filter and explore trafficking volume by different species classes. You will modify the given file: `wrangling_scripts/Q5.py`

Technology	• Python 3.10.x • Flask
Allowed Libraries	Python standard libraries Libraries already imported in <code>Q5.py</code>
Deliverables	Q5.py : Completed Python file with your changes

Tasks and point breakdown

1. **username()** - Update the `username()` method inside `Q5.py` by including your GT username.
2. Install Flask on your machine by running `$ pip install Flask`
 - a. You can optionally create a virtual environment by following the steps [here](#). Creating a virtual environment is purely optional and can be skipped.
3. To run the code, navigate to the Q5 folder in your terminal/command prompt and execute the following command: `python run.py`. After running the command, go to <http://127.0.0.1:3001/> on your browser. This will open `index.html`, showing a table in which the rows returned by `data_wrangling()` are displayed. **After completing the question**, you can then choose different species classes from the dropdown and see how the data table updates dynamically.
4. You must solve the following two sub-questions:
 - a. **[2 pts]** Generate a list of unique species **classes** for options in the dropdown menu. Sort the list alphabetically.
 - b. **[3 pts]** Filter, sort, and limit the data
 - i. First, filter the data to only the specified species class. If no class is specified, include all the data. You should still sort and limit if no class is specified.
 - ii. Next, sort the data by the count column in descending order. You do not need to worry about tiebreaks.
 - iii. Last, limit the data to only the top 10 rows. If the number of rows is fewer than 10 then return all rows.